

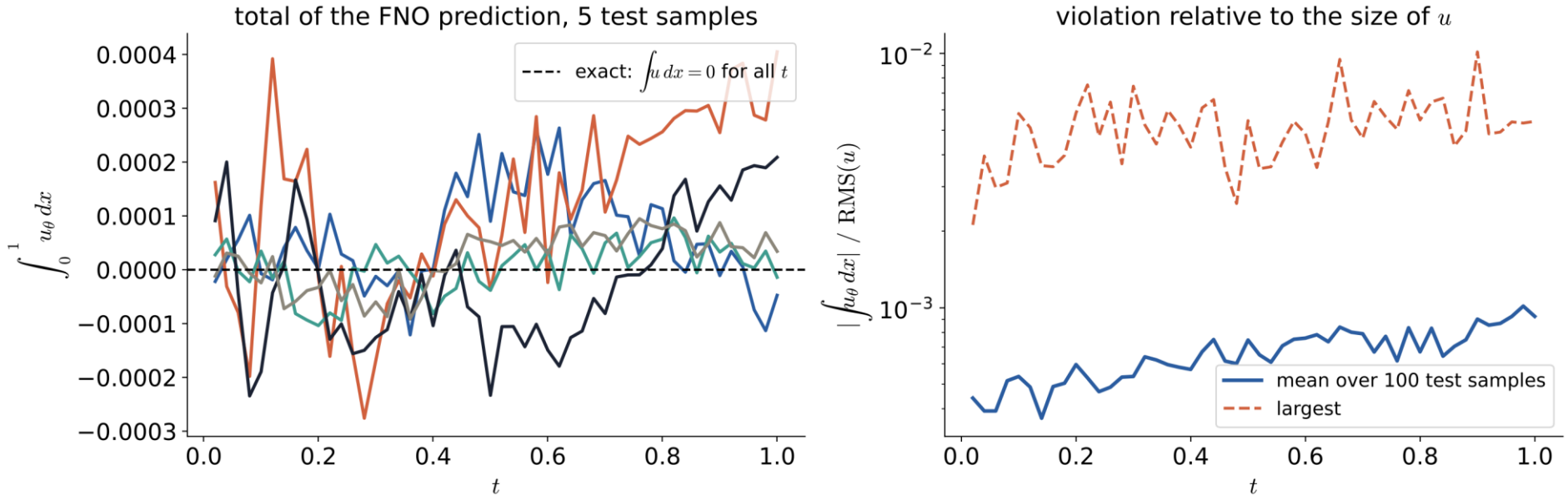
ML FOR SCIENCE · LECTURE 15

Physics-Constrained and Hybrid Learning

Aparna Chandramowlishwaran · EECS 195/221 · Fall 2026

Learned models satisfy physics approximately

- Periodic Burgers conserves the total: $\int_0^1 u\,dx$ is the same at all times (here 0)
- Lecture 12 FNO: the total of its prediction is **small but not zero**, and the relative violation grows with t
- Lecture 14 PINN: the boundary condition is a penalty, so $u(\pm 1, t) = 0$ holds only approximately



Today: hard constraints, and hybrid models that combine a numerical solver with a network

Soft and hard constraints

- A constraint $\mathcal{C}(u) = 0$: boundary condition, conserved total, $\nabla \cdot \mathbf{u} = 0$, symmetry
- **Soft constraint**: penalty with weight λ, satisfied approximately and only at the sampled points:

$$\min_{\theta} \; \mathcal{L}_{\text{data}}(\theta) + \lambda \left\| \mathcal{C}(u_\theta) \right\|^2$$

- **Hard constraint**: model satisfies $\mathcal{C}$ for **all** weights θ, during and after training:

$$\min_{\theta} \; \mathcal{L}_{\text{data}}(\theta) \quad \text{subject to} \quad \mathcal{C}(u_\theta) = 0 \;\; \text{for all } \theta$$

- Three ways to build a hard constraint:
  - **Architecture**: write the output so the constraint holds (distance functions, fluxes, stream functions, Hamiltonians)
  - **Projection**: map the network output to the nearest function that satisfies the constraint
  - **Hybrid**: numerical method that satisfies the constraint, with a learned correction inside it
- Lecture 5: equivariant CNNs were hard constraints for symmetry. Augmentation was the soft version

Karniadakis, Kevrekidis, Lu, Perdikaris, Wang & Yang, Physics-informed machine learning, Nature Reviews Physics 2021

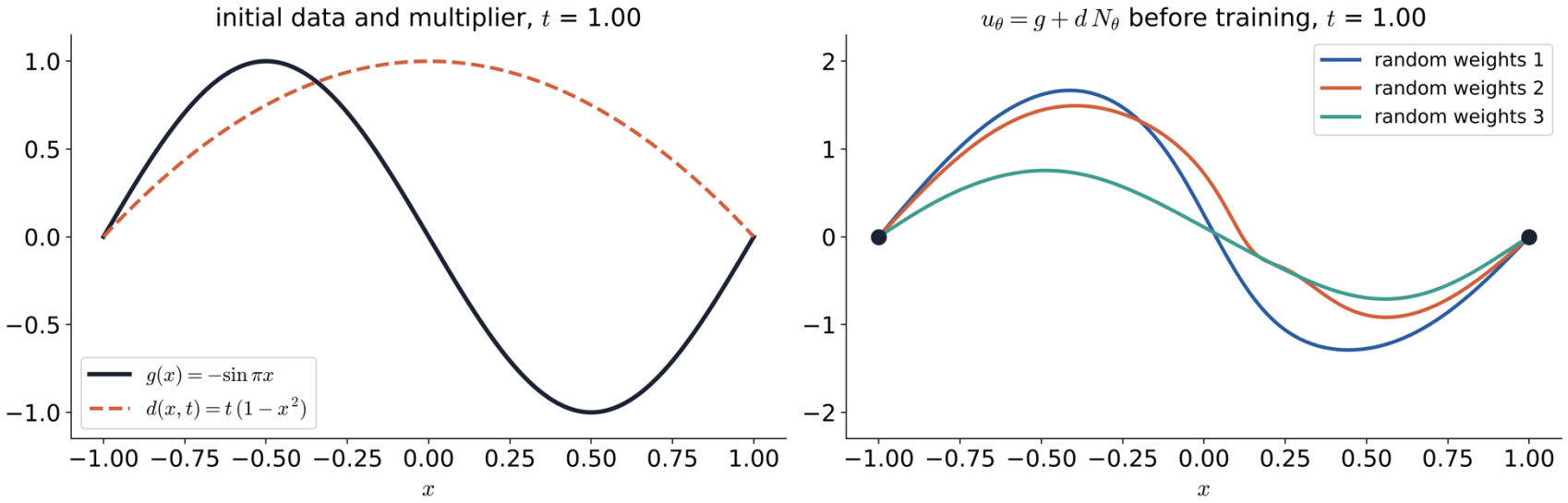
Boundary and initial conditions by construction

- Write the output as

$$u_\theta(x,t) = g(x,t) + d(x,t)\, N_\theta(x,t)$$

- g satisfies the conditions; $d = 0$ on the boundary and at $t = 0$; N_θ is an unconstrained network. Then $u_\theta = g$ on the boundary and at $t = 0$ for all θ, and the loss is the **PDE residual** of u_θ
- Burgers: $-\sin(\pi x)$ is the initial condition and is 0 at $x = \pm 1$:

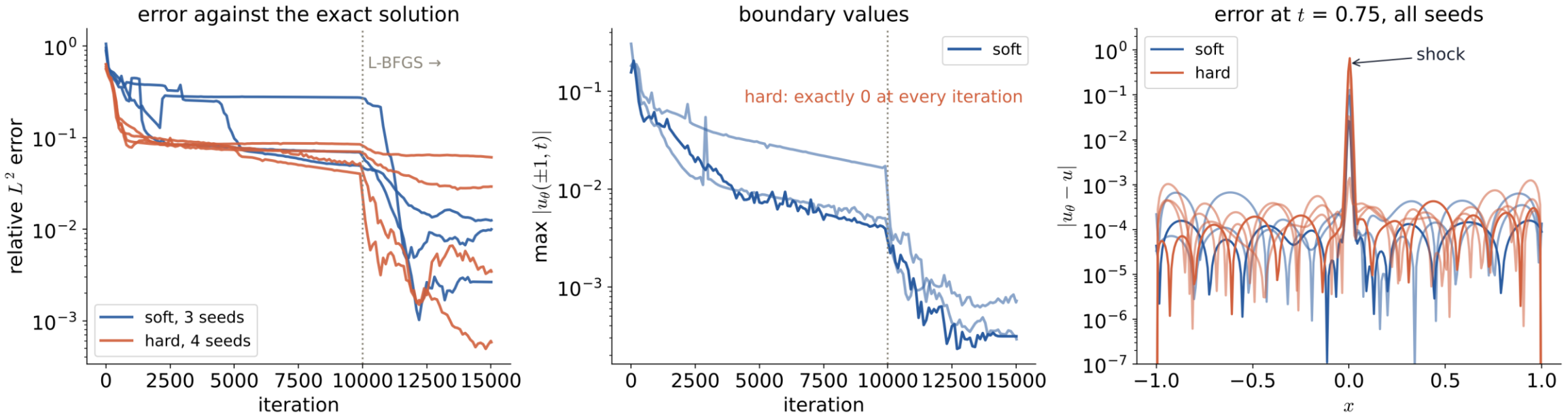
$$u_\theta(x,t) = -\sin(\pi x) + t\,(1 - x^2)\, N_\theta(x,t)$$



Lagaris, Likas & Fotiadis, Artificial neural networks for solving ordinary and partial differential equations, IEEE Trans. Neural Networks 1998

Exact conditions in the Burgers PINN

- Same network, collocation points and optimizer as in Lecture 14. Only the output layer differs
- Soft: the boundary values are **small but never zero**. Hard: they are **zero for all weights**
- Final error varies across seeds for both. Away from the shock both are near 10^{-4}; the **shock** dominates (right)



Exact conditions remove loss terms and their weights. PDE residual remains the hard part

Conservation laws and conserved totals

- Recall: in a conservation law, the change of u in a region equals the net flux F through its boundary. Burgers:

$$\frac{\partial u}{\partial t} + \frac{\partial F}{\partial x} = 0, \qquad F = \frac{u^2}{2} - \nu \frac{\partial u}{\partial x}$$

- Integrate over a periodic domain. The boundary fluxes are equal, so the total is constant:

$$\frac{d}{dt} \int_0^1 u \, dx = -\big[F(1) - F(0)\big] = 0$$

- In fluids: mass, momentum and energy. The shock speed follows from conservation (Rankine-Hugoniot). A method that does not conserve can converge to a shock at the **wrong position**
- Next-step network $u^{n+1} = u^n + N_\theta(u^n)$ changes the total by $\Sigma_i N_{\theta,i}$ (nothing makes that zero)
- Finite-volume methods conserve exactly because they update cells with fluxes

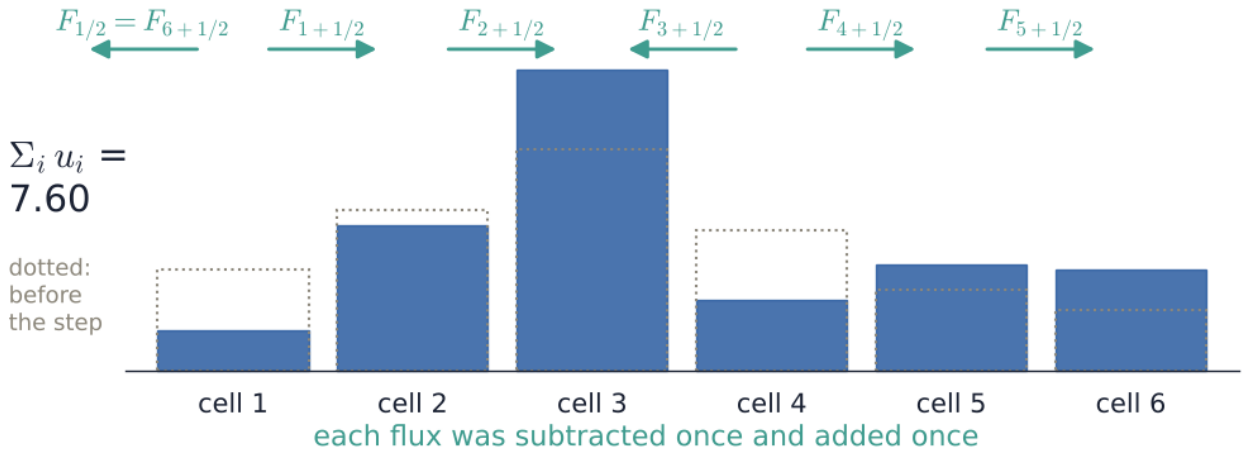
LeVeque, Finite Volume Methods for Hyperbolic Problems, Cambridge 2002

Finite-volume form conserves for any flux

- Finite-volume update, with u_i the average of u over cell i and $F_{i+1/2}$ the flux through the face between cells i and i+1:

$$u_i^{n+1} = u_i^n - \frac{\Delta t}{\Delta x}\left(F_{i+1/2} - F_{i-1/2}\right) \quad \Longrightarrow \quad \sum_{i=1}^{N} u_i^{n+1} = \sum_{i=1}^{N} u_i^n$$

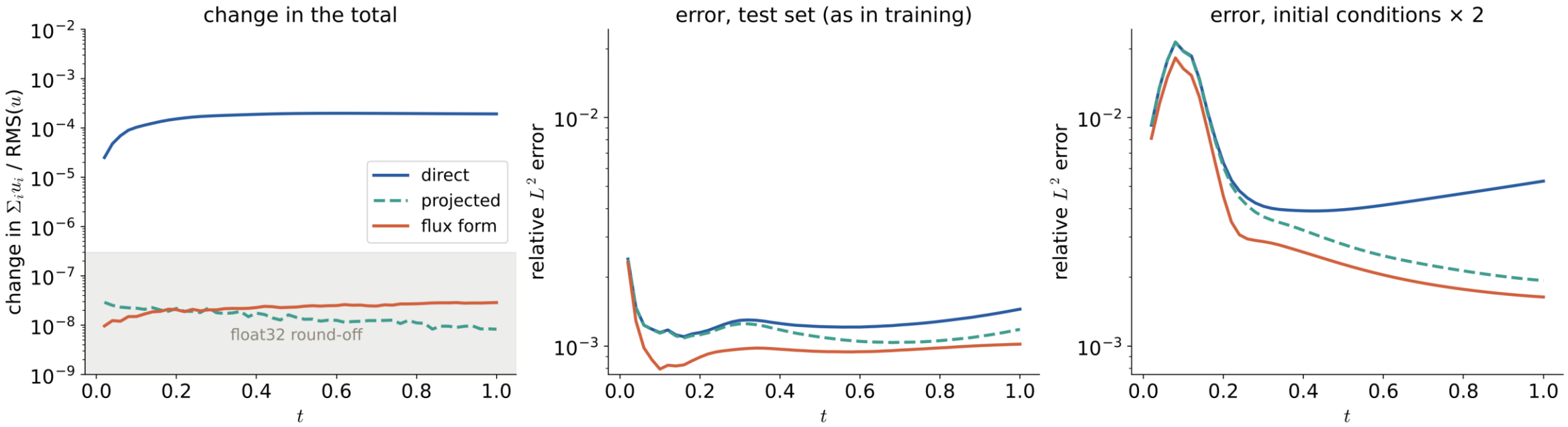
- Each face flux is subtracted from one cell and added to its neighbor, so the total is unchanged for **any** values of F



LeVeque, Finite Volume Methods for Hyperbolic Problems, Cambridge 2002

A flux-form network on a coarse Burgers grid

- Burgers (Lecture 12 data) averaged onto 32 cells; one network step = $\Delta t = 0.02$; rollout for 50 steps
- **Direct**: $u^{n+1} = u^n + N_\theta(u^n)$. **Flux form**: same CNN outputs $F_{i+1/2}$. **Projected**: direct, then subtract the mean change
- Direct: the total drifts. Flux form and projection: **round-off** (left). Similar error on the test set but with initial conditions twice as large, the conservative models have **lower error** late in the rollout



Projection onto a linear constraint

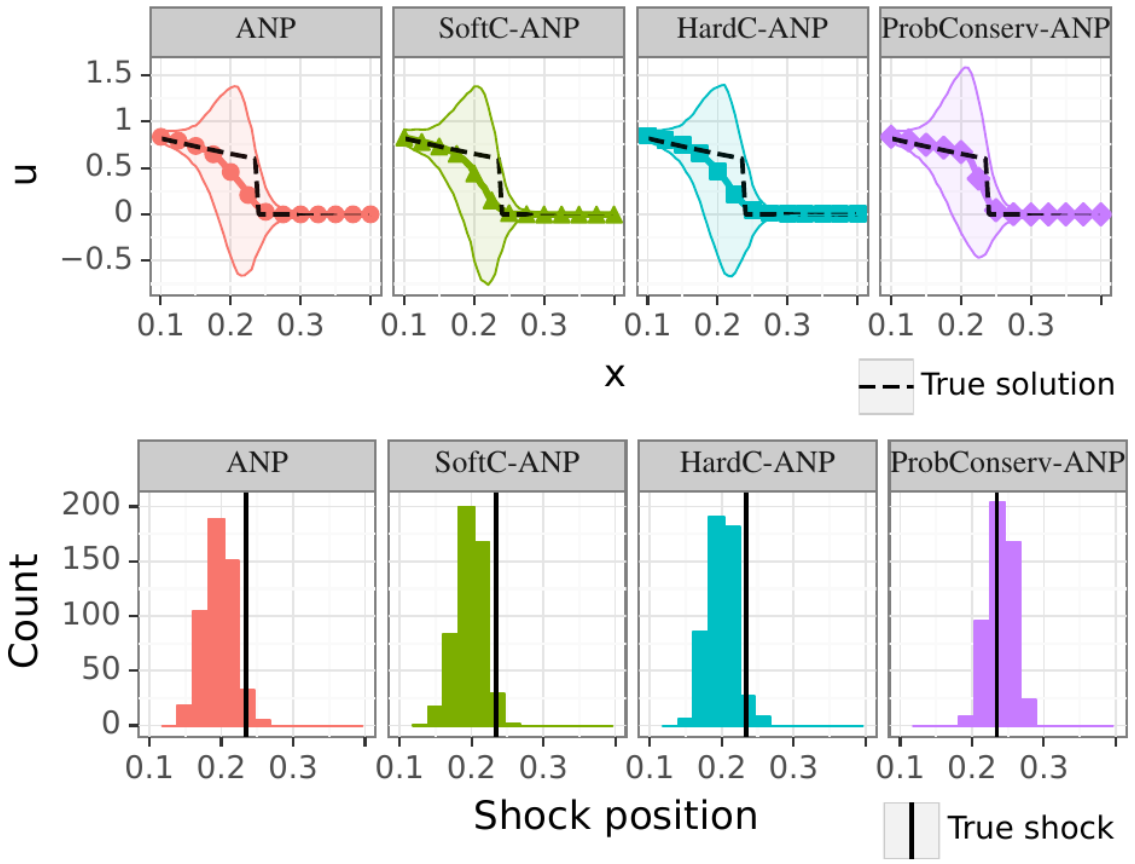
- A linear constraint $Gu = b$; a conserved total is one row of G. Projection gives the **closest** state that satisfies it:

$$\tilde{u} = u - G^{\mathsf{T}}\left(GG^{\mathsf{T}}\right)^{-1}(Gu - b)$$

- Conserved total: subtract the mean error from each cell

$$\tilde{u}_i = u_i - \frac{1}{N}\sum_{j=1}^{N}\left(u_j - u_j^0\right)$$

- ProbConserv: the model predicts a mean and a **variance**; the update moves the prediction most where it is most uncertain
- Stefan problem with a moving shock (right): penalty and uniform projection place the shock too far left; the variance-weighted update places it correctly



Hansen, Maddix, Alizadeh, Gupta & Mahoney, Learning physical models that can respect conservation laws, ICML 2023

Divergence-free velocity from a stream function

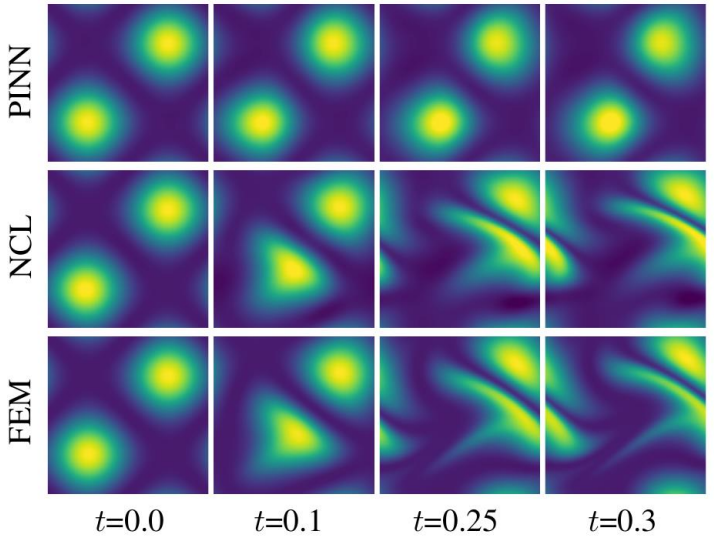
- Incompressible flow: $\nabla \cdot \mathbf{u} = 0$. The network outputs a **stream function** $\psi_\theta(x, y)$, and autodiff gives

$$u = \frac{\partial \psi_\theta}{\partial y}, \qquad v = -\frac{\partial \psi_\theta}{\partial x}$$

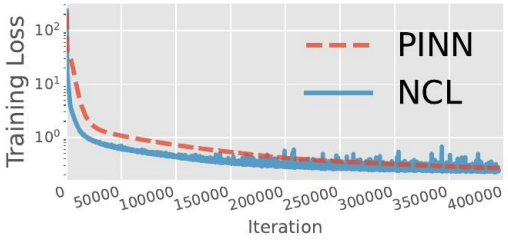
- Mixed derivatives commute, so for all θ

$$\nabla \cdot \mathbf{u} = \frac{\partial^2 \psi_\theta}{\partial x \, \partial y} - \frac{\partial^2 \psi_\theta}{\partial y \, \partial x} = 0$$

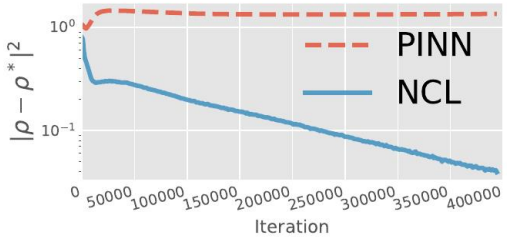
- 3D: $\mathbf{u} = \nabla \times \mathbf{A}$ with a vector potential $\mathbf{A}_\theta$
- Neural conservation laws: density and velocity of the continuity equation from one divergence-free field
- Incompressible Euler (right): PINN fits the initial density but not the advection. NCL model follows a finite-element reference



(a) Learned density function



(b) Convergence of training loss



(c) Comparison to FEM solution

Richter-Powell, Lipman & Chen, Neural conservation laws: a divergence-free perspective, NeurIPS 2022

Hamiltonian neural networks conserve energy

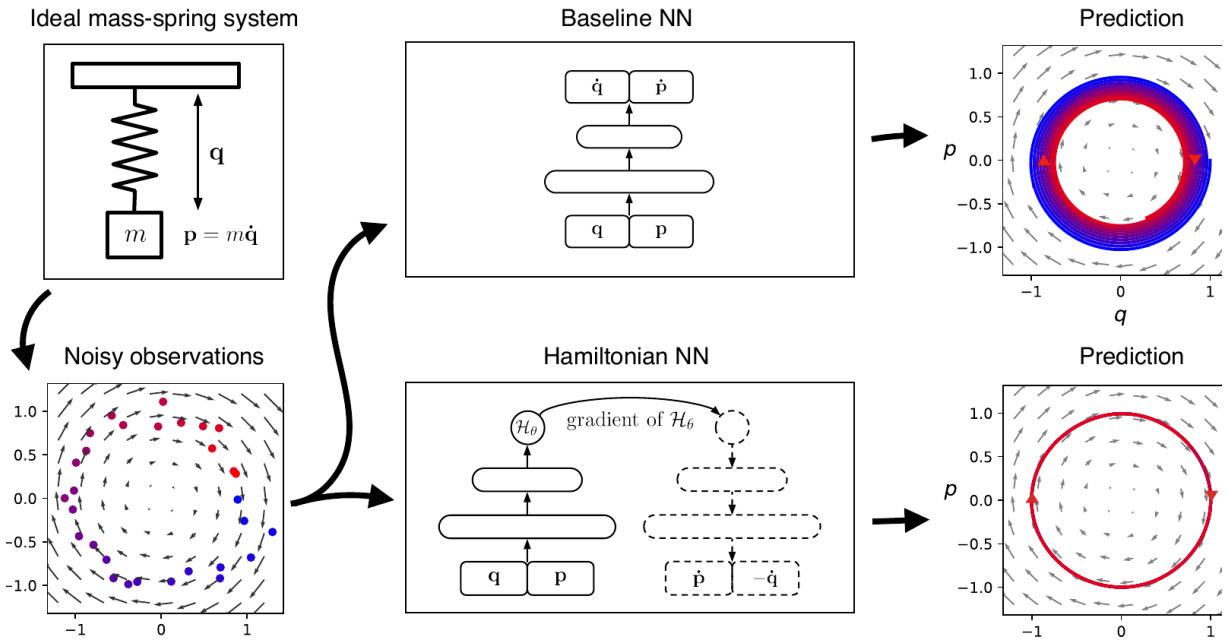
- A neural ODE learns $dz/dt = f_\theta(z)$. Nothing enforces constant energy
- HNN: the network outputs one scalar, the energy $H_\theta(q, p)$, and the dynamics are

$$\frac{dq}{dt} = \frac{\partial H_\theta}{\partial p}, \qquad \frac{dp}{dt} = -\frac{\partial H_\theta}{\partial q}$$

- By the chain rule, for all θ

$$\frac{dH_\theta}{dt} = \frac{\partial H_\theta}{\partial q}\frac{\partial H_\theta}{\partial p} - \frac{\partial H_\theta}{\partial p}\frac{\partial H_\theta}{\partial q} = 0$$

- Only the time integrator adds error. Mass-spring system from noisy data (right): the baseline spirals out; the HNN stays on a closed orbit
- Lagrangian neural networks do the same with $L_\theta(q, \dot{q})$



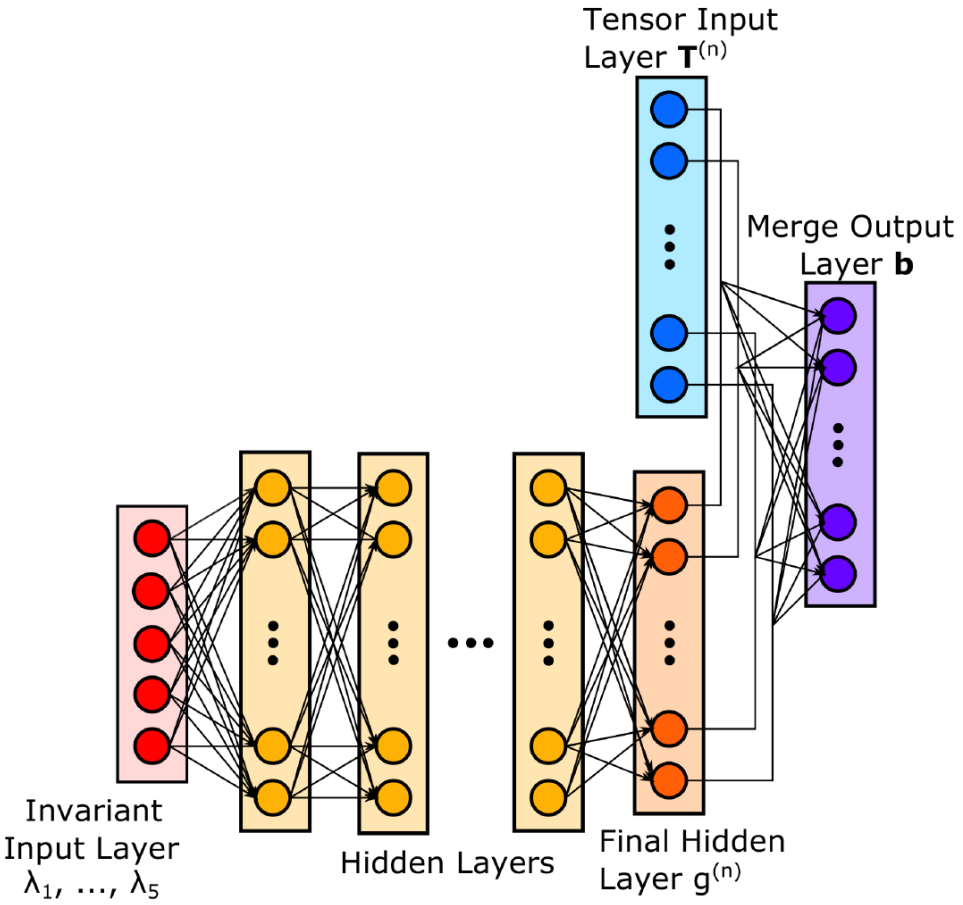
Greydanus, Dzamba & Yosinski, Hamiltonian neural networks, NeurIPS 2019. Cranmer et al., Lagrangian neural networks, ICLR workshop 2020

Tensor basis networks for turbulence closures

- RANS models need the Reynolds stress anisotropy **b** from the mean strain rate **S** and rotation rate **R**
- **b** must rotate with the coordinate frame (Lecture 5: equivariance)
- Any such **b** is a combination of 10 tensors built from **S** and **R**, with coefficients that depend only on 5 **invariants** (Pope 1975):

$$\mathbf{b} = \sum_{n=1}^{10} g^{(n)}(\lambda_1, \ldots, \lambda_5)\, \mathbf{T}^{(n)}(\mathbf{S}, \mathbf{R})$$

- TBNN: an MLP maps the invariants to $g^{(n)}$; the last layer multiplies by the tensor basis. Invariance holds for all θ
- More accurate than a generic MLP on the same inputs in duct flow and flow over a wavy wall



Ling, Kurzawski & Templeton, Reynolds averaged turbulence modelling using deep neural networks with embedded invariance, JFM 2016

Choosing how to enforce a constraint

	Soft (penalty)	Hard (architecture)	Projection
Constraint holds	approximately, at sampled points	**exactly**, for all weights	**exactly**, after each output
Training	extra loss terms and weights λ	data or residual loss only	train with or without the projection
Effort	write the violation as a loss	a construction for each constraint	solve a small (often linear) problem
Limits	trade-off with accuracy	Neumann conditions, complex geometry; may slow optimization	linear constraints are easy; nonlinear need an iterative solve
Examples	PINN loss, PINO	flux form, ψ, HNN, TBNN	mean correction, ProbConserv

- Enforce a constraint exactly only when it is **certain**: a wrong hard constraint cannot be undone by data
- Hard and soft combine: exact boundary conditions plus a residual penalty is the usual PINN setup

Négiar, Mahoney & Krishnapriyan, Learning differentiable solvers for systems with hard constraints, ICLR 2023

Hybrid models combine a solver and a network

- One hybrid step: a numerical solver $\mathcal{S}_\Delta$ on a coarse grid, then a learned correction N_θ

$$u^{n+1} = \mathcal{S}_\Delta(u^n) + N_\theta\big(\mathcal{S}_\Delta(u^n)\big)$$

- Continuous form: known terms plus a learned term, integrated with a standard method

$$\frac{du}{dt} = f_{\text{known}}(u) + N_\theta(u)$$

- A coarse grid under-resolves fronts and small scales. A fine solver is accurate but expensive
- The solver supplies conservation, boundary conditions and the known physics. The network learns the **missing part**: a **closure**, **discretization coefficients**, or a **correction** of the state after each step

Pure solver ← hybrid → pure ML: hybrids trade some speed for stability and generalization

Closure models for unresolved scales

- Average Burgers over a cell. The nonlinear term leaves the **subgrid flux** τ:

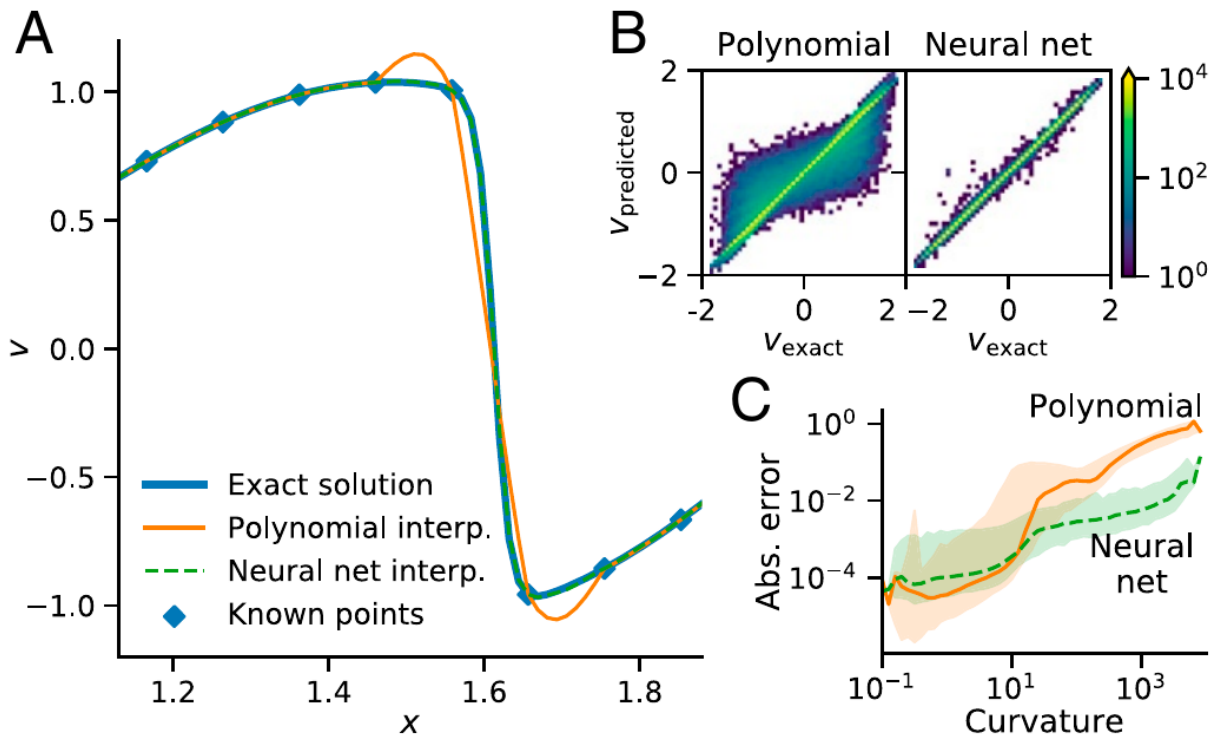
$$\frac{\partial \bar{u}}{\partial t} + \frac{\partial}{\partial x}\left(\frac{\bar{u}^2}{2}\right) = \nu \frac{\partial^2 \bar{u}}{\partial x^2} - \frac{\partial \tau}{\partial x}, \qquad \tau = \frac{1}{2}\left(\overline{u^2} - \bar{u}^2\right)$$

- τ depends on the unresolved u. The coarse state does not determine it: a model must close the equation (large-eddy simulation, RANS)
- Classical closures: Smagorinsky, an eddy viscosity $\nu_t \propto \Delta x^2 |\partial \bar{u}/\partial x|$; usually too dissipative
- Learned closure: $N_\theta(\bar{u}) \approx \tau$, trained on filtered high-resolution simulations
- **A priori** training fits τ at each time independently. The closure then runs inside the solver for many steps, on its own errors

Duraisamy, Iaccarino & Xiao, Turbulence modeling in the age of data, Annual Review of Fluid Mechanics 2019

Learned discretizations

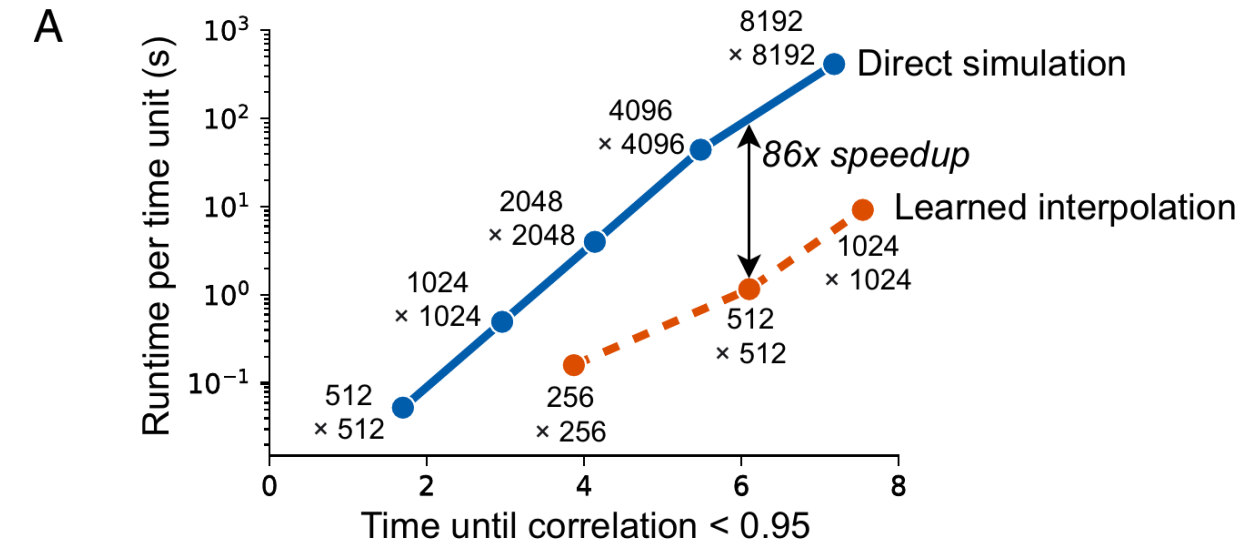
- A finite-difference stencil approximates $\partial v/\partial x \approx \Sigma_k \alpha_k v_{i+k}$ from neighboring cell values. Standard α come from fitting a polynomial
- Learn the α_k with a CNN that sees the local solution. The update stays in **finite-volume** form, so the total is conserved
- Polynomial interpolation overshoots near a shock (A). The network interpolates accurately (B), with smaller error at high curvature (C)
- Burgers, KdV and Kuramoto-Sivashinsky: accurate on grids **4-8x coarser** than standard schemes

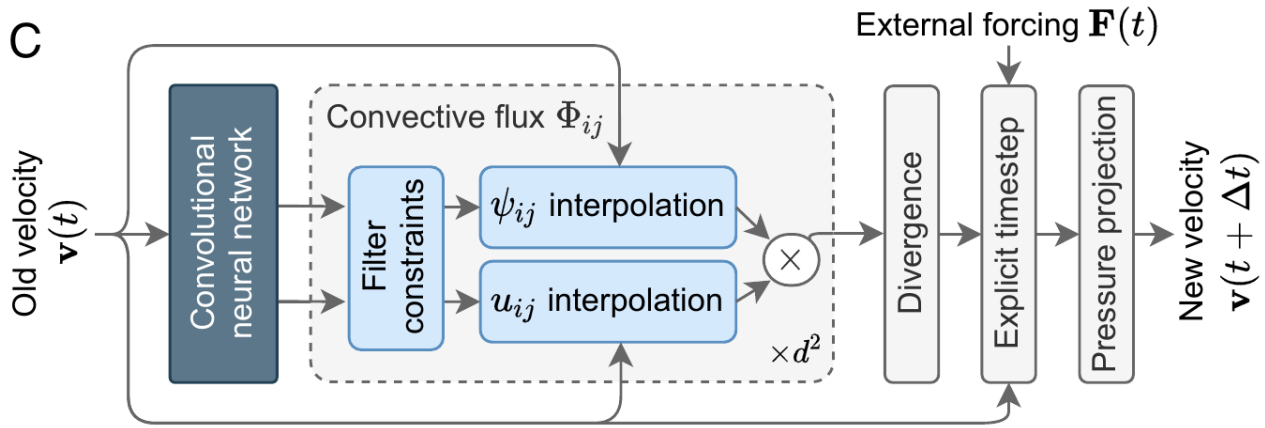


Bar-Sinai, Hoyer, Hickey & Brenner, Learning data-driven discretizations for partial differential equations, PNAS 2019

Learned interpolation in a CFD solver

- 2D turbulence (Kolmogorov flow). A differentiable Navier-Stokes solver; a CNN outputs interpolation coefficients for the convective flux
- Divergence and pressure projection stay in the solver: incompressibility is exact
- Matches direct simulation on a grid 8-10x finer in each dimension, **40-80x faster**
- Generalizes to larger domains, decaying turbulence and higher Reynolds numbers





Kochkov, Smith, Alieva, Wang, Brenner & Hoyer, Machine learning-accelerated computational fluid dynamics, PNAS 2021

Training through the solver

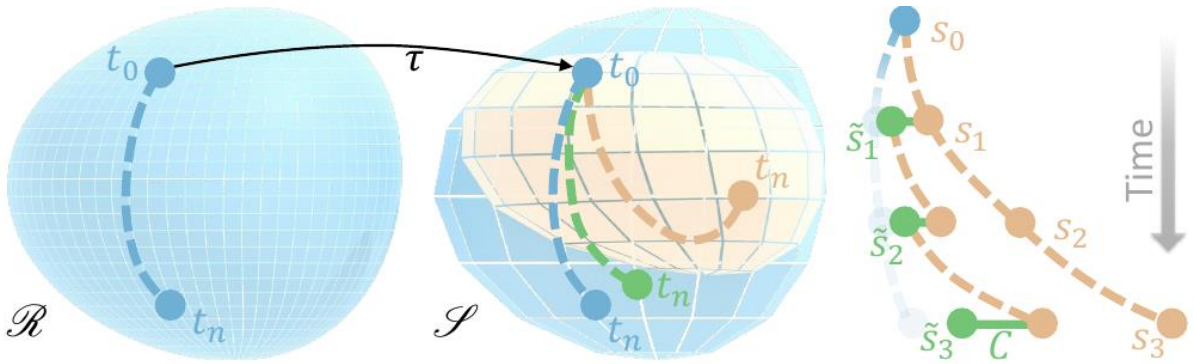
- M_θ: one hybrid step, solver plus correction; M_θ^k: k steps in a row; u^n: reference states
- **A priori**: one step from reference data. The model never sees its own errors

$$\mathcal{L}_{\text{a priori}} = \left\| u^{n+1} - M_\theta(u^n) \right\|^2$$

- **A posteriori** (solver-in-the-loop): unroll m steps and backpropagate through the solver (Lecture 7: unrolled training). Needs a **differentiable solver** (JAX-CFD, PhiFlow) or an adjoint

$$\mathcal{L}_{\text{a posteriori}} = \sum_{k=1}^{m} \left\| u^{n+k} - M_\theta^k(u^n) \right\|^2$$

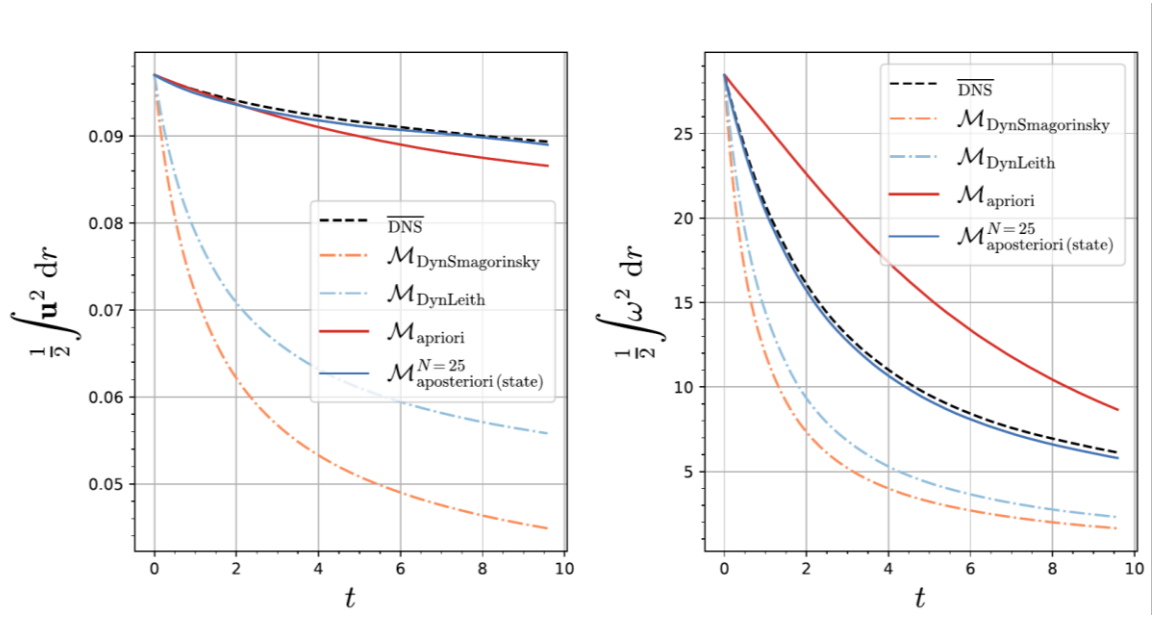
- Below: reference ($\mathcal{R}$, blue) and coarse ($\mathcal{S}$, orange) trajectories; the correction (green) pulls each step back



Um, Brand, Fei, Holl & Thuerey, Solver-in-the-loop: learning from differentiable physics to interact with iterative PDE-solvers, NeurIPS 2020

A posteriori training in 2D turbulence

- Subgrid closure for quasi-geostrophic turbulence (ocean and atmosphere models); decaying turbulence over 10 time units
- Energy (left) and enstrophy (right) of the coarse simulation. Black dashed: filtered DNS
- Smagorinsky and Leith: too dissipative. A priori network: drifts. **A posteriori** network (N = 25 steps): follows DNS

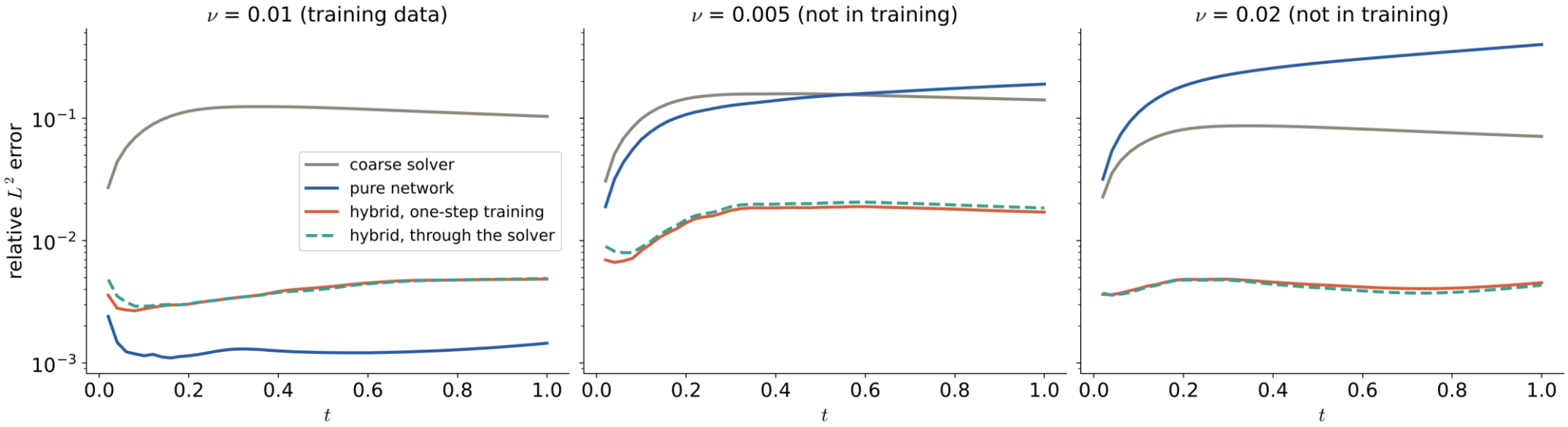


Train the closure on the states it will see: its own trajectories inside the solver

Frezat, Le Sommer, Fablet & Balarac, A posteriori learning for quasi-geostrophic turbulence parametrization, JAMES 2022

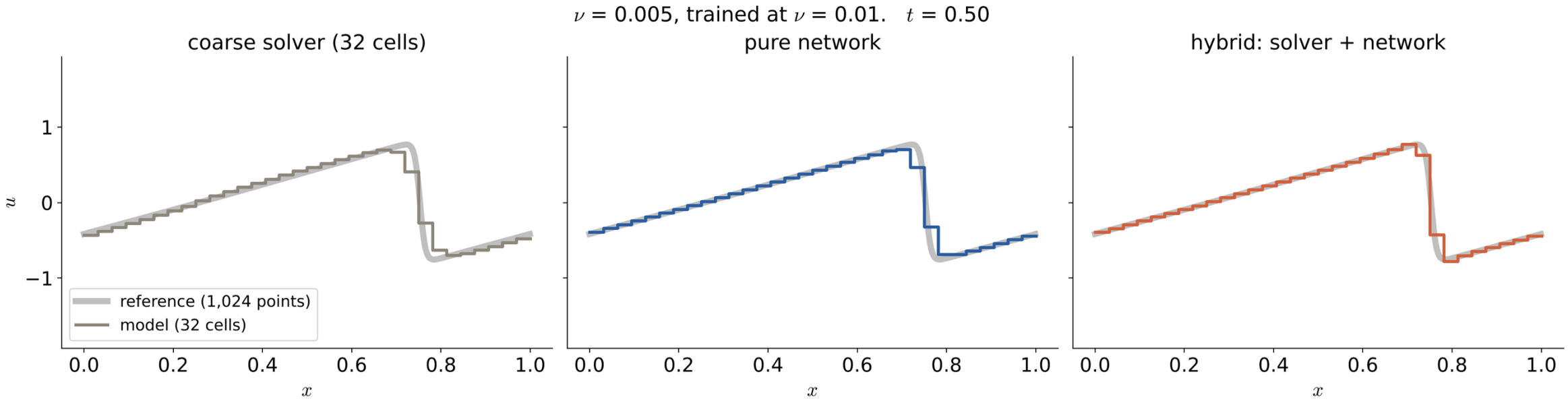
Hybrid models on the coarse Burgers grid

- Coarse solver: first-order finite volume on 32 cells; smears fronts. Hybrid: solver step + learned flux correction
- v = 0.01 as in training: the hybrid reduces the solver error by more than 10x. The pure network is **the most accurate**
- v = 0.005 and 0.02 (the solver gets the new v; networks unchanged): the pure network fails, the **hybrids stay accurate**
- One-step and through-the-solver training give the same error here: the one-step hybrid is already stable



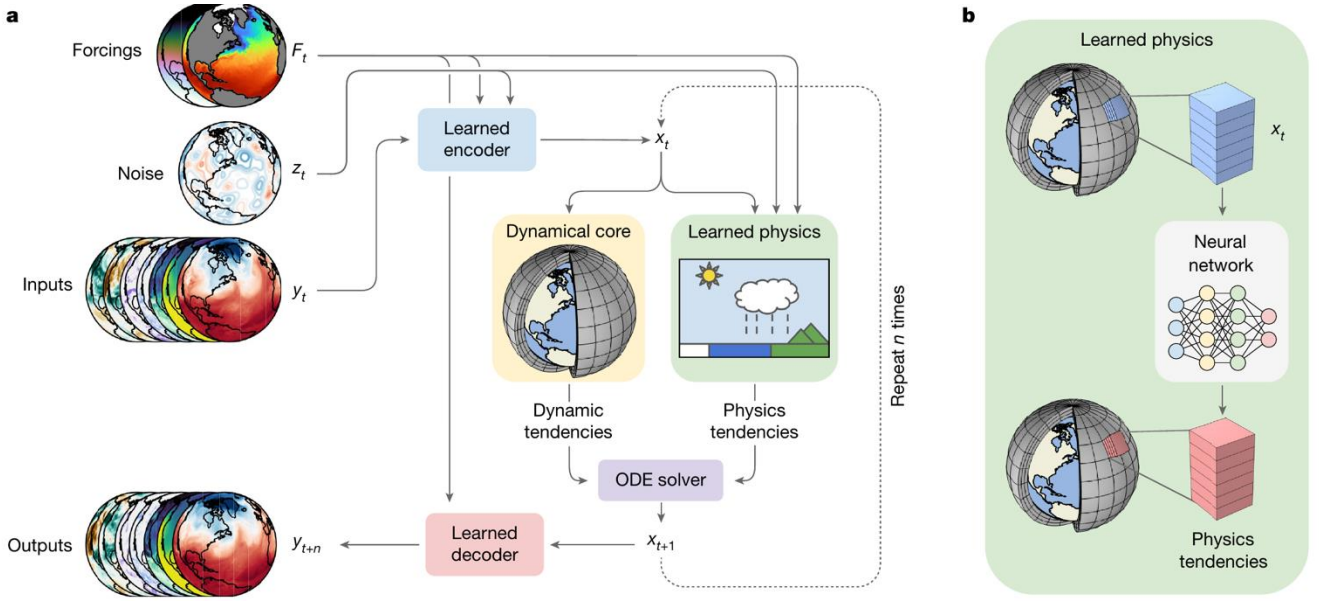
Rollouts at a viscosity outside the training data

- One test sample at v = 0.005; all models trained at v = 0.01. Gray: reference on 1,024 points. Colors: models on 32 cells
- Coarse solver: the front is smeared. Pure network: it dissipates as if v = 0.01, so the peak decays too fast and the front is too wide. Hybrid: **sharp front**, correct amplitude



A hybrid model for weather and climate

- NeuralGCM: a differentiable **dynamical core** solves the equations of atmospheric motion. Network contributes the **learned physics**: clouds, radiation, precipitation, subgrid processes
- The network acts on each vertical column (b). ODE solver combines both tendencies (a)
- Trained **through the solver** on rollouts up to 5 days, against reanalysis data (ERA5)
- 1-15 day forecasts comparable to the best physics-based and ML models. Stable multi-decade climate runs at coarse resolution



Kochkov et al., Neural general circulation models for weather and climate, Nature 2024

Constraints and hybrids in your project

- List what must hold exactly in your problem: boundary values, conserved totals, $\nabla \cdot \mathbf{u} = 0$, symmetries, positivity
- Cheap first steps:
  - predict fluxes or increments in conservative form; subtract the mean error (projection)
  - exact boundary conditions in a PINN with $g + dN_\theta$; periodic inputs for periodic domains
  - a stream-function output for 2D incompressible velocity
- Hybrid: start from a coarse solver you can write in PyTorch or JAX (1D/2D finite volume) and learn a correction
- Train one step first, then unroll through the solver. Report error against a fine reference **and** the cost
- Baselines: the same network without the constraint; the coarse solver alone; a solver at equal accuracy
- BubbleML: mass and energy are conserved away from the interface; the coarse 48 x 48 data are a natural target for a hybrid with a learned correction

JAX-CFD: github.com/google/jax-cfd. PhiFlow: github.com/tum-pbs/PhiFlow. torchdiffeq: github.com/rtqichen/torchdiffeq

Summary

- **Soft** constraints (penalties) hold approximately. **Hard** constraints hold for all weights
- Exact boundary and initial conditions: $u_\theta = g + d\,N_\theta$ with a distance function d
- Conservation: predict face fluxes (finite-volume form) or project onto the conserved total
- Divergence-free velocity from a stream function; energy from a learned Hamiltonian; invariance from a tensor basis
- Hard-code only what is certain; combine with soft penalties for the rest
- **Hybrid** models combine a coarse solver with learned closures, stencil coefficients or corrections
- Train hybrids **through the solver** (a posteriori) when one-step training drifts in rollout
- Examples: learned discretizations, learned interpolation in CFD, NeuralGCM

After class

- Kochkov et al., Machine learning-accelerated computational fluid dynamics, PNAS 2021
- Um et al., Solver-in-the-loop, NeurIPS 2020: Sections 1-3
- Greydanus, Dzamba & Yosinski, Hamiltonian neural networks, NeurIPS 2019
- Hansen et al., Learning physical models that can respect conservation laws, ICML 2023: Sections 1-3
- Optional: Kochkov et al., Neural general circulation models for weather and climate, Nature 2024
- Optional: Duraisamy, Iaccarino & Xiao, Turbulence modeling in the age of data, Annu. Rev. Fluid Mech. 2019

NEXT: HANDS-ON

From concepts to code

→ **Open the Lecture 15 notebook**

1. The total of a Lecture 12 FNO prediction
2. Exact boundary and initial conditions in Burgers PINN
3. Direct, projected and flux-form networks on a coarse grid: rollout error and the conserved total
4. Coarse solver, solver + correction trained one step, solver + correction trained through the solver
5. Viscosity outside the training data: pure network vs. hybrid